M01 — Fundamentos de ASP.NET Core MVC

Índice del módulo

1.1. Introducción al desarrollo web con ASP.NET Core

1.2. Arquitectura Model-View-Controller

1.3. Estructura, convenciones y puesta en marcha de un proyecto ASP.NET Core MVC

1.4. Controladores, acciones, parámetros y routing en ASP.NET Core MVC

1.5. Vistas Razor, modelos tipados, layouts y vistas parciales

Material teórico

Índice del módulo

- **1.1. Introducción al desarrollo web con ASP.NET Core**
- **1.2. Arquitectura Model-View-Controller**
- **1.3. Estructura de un proyecto ASP.NET Core MVC**
- **1.4. Controladores, acciones y routing**
- **1.5. Vistas Razor, modelos tipados, layouts y vistas parciales**

1.1. Introducción al desarrollo web con ASP.NET Core

Objetivos de aprendizaje

Al finalizar este punto, el alumno podrá explicar qué función desempeña ASP.NET Core en una aplicación web, diferenciar una respuesta HTML generada por MVC de una respuesta JSON generada por una API, identificar los elementos esenciales de una petición HTTP y seguir la ejecución desde el navegador hasta una acción MVC y una vista Razor.

Se utilizará una primera aplicación del curso, `CursoWeb.Mvc`, que mostrará una página relacionada con el futuro catálogo de productos. No necesitaremos base de datos, autenticación ni conocimientos nuevos de C#.

1.1.1. Del conocimiento de Web API a una aplicación web visible

Los alumnos ya conocen ASP.NET Core Web API: una aplicación recibe una petición como `GET /api/productos`, ejecuta un endpoint y devuelve, habitualmente, JSON. Un programa consumidor interpreta después esos datos. Cuando queremos construir una **interfaz web tradicional**, el servidor también puede generar la representación HTML que ve el navegador.

ASP.NET Core es la plataforma de desarrollo web de .NET. Facilita la creación de aplicaciones que atienden HTTP y ofrece infraestructura común: servidor HTTP, enrutamiento, servicios, configuración y una secuencia de procesamiento de las peticiones. Encima de esa plataforma se utilizan distintos estilos de programación: controladores de API, MVC con vistas, Razor Pages y componentes Blazor. Comparten plataforma, pero no son intercambiables.

Para situarnos, imaginemos que un usuario abre en el navegador la dirección del catálogo. La aplicación debe decidir qué recurso se ha solicitado, preparar la información de los productos y producir una respuesta. En una API, el resultado puede ser una colección JSON. En MVC, el resultado habitual de una acción que devuelve una vista es un documento HTML listo para ser interpretado y mostrado por el navegador.

Por ejemplo, una API podría producir una respuesta semejante a esta:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[{"id":1,"nombre":"Cuaderno"},{"id":2,"nombre":"Lápiz"}]
```

Una acción MVC que devuelve una vista podría producir este otro contenido:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="es">
  <body>
    <h1>Productos</h1>
    <ul><li>Cuaderno</li><li>Lápiz</li></ul>
  </body>
</html>
```

Ambos ejemplos utilizan HTTP. La diferencia esencial es el **contrato de presentación**: datos estructurados destinados a un consumidor, frente a marcado preparado para representarse como

página. No es una separación absoluta: una API puede devolver otros formatos y una aplicación MVC también puede devolver JSON. Es la forma de trabajar que utilizaremos en el curso la que establece esa comparación.

Pregunta de razonamiento. Si una API devuelve el nombre y el precio de un producto, ¿por qué el navegador no presenta automáticamente una ficha de producto con el formato deseado? Porque JSON describe datos, pero no define por sí mismo los elementos HTML, su estructura visual ni los estilos. Una interfaz necesita construir esa representación, y MVC puede hacerlo del lado del servidor.

1.1.2. Qué ocurre al escribir una URL

Cuando introducimos `http://localhost:5091/` en el navegador, este actúa como **cliente HTTP**. En nuestro laboratorio, `localhost` identifica el propio ordenador, el puerto `5091` selecciona el proceso servidor y `/` es la ruta solicitada. En un sitio público intervendrían además resolución de nombres, red y, normalmente, HTTPS. Estos elementos no alteran la lógica básica de cliente, petición y respuesta.

Una petición HTTP contiene, como mínimo en términos conceptuales:

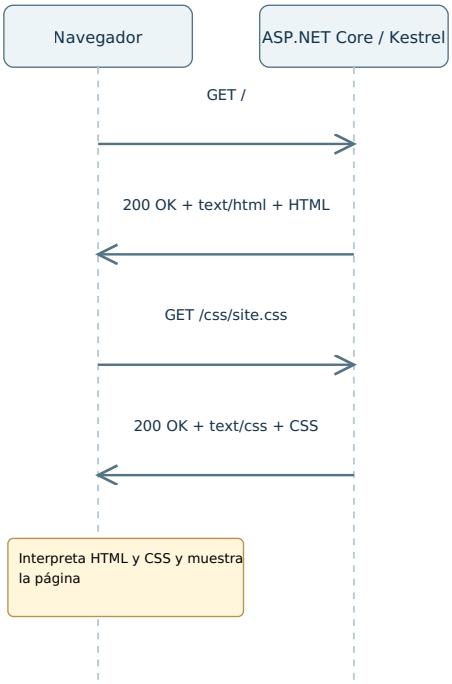
- **Método.** Indica el tipo de operación solicitada. En este punto utilizaremos `GET` para consultar una página. El día 2 utilizaremos `POST` para enviar formularios.
- **Destino.** La URL y el camino o *path* que el servidor examina, por ejemplo `/`, `/Home/Index` o `/css/site.css`.
- **Cabeceras.** Metadatos como los formatos que acepta el cliente. No son el cuerpo de la página.
- **Cuerpo.** Datos opcionales asociados a la petición; normalmente una navegación GET sencilla no necesita uno.

La respuesta también tiene partes diferenciadas:

- **Código de estado.** `200 OK` significa que la solicitud se atendió satisfactoriamente; `404 Not Found` significa que el servidor no encontró un recurso o ruta compatible.
- **Cabeceras de respuesta.** Por ejemplo, `Content-Type` indica el tipo del contenido devuelto.
- **Cuerpo de respuesta.** En nuestro ejemplo, el documento HTML de la vista Razor.

Una carga de página puede producir **varias peticiones HTTP**. El navegador primero recibe el HTML; si este enlaza una hoja de estilo con `<link rel="stylesheet" ...>`, solicita además el archivo CSS. Por eso, cuando abrimos la pestaña *Red/Network* de las herramientas de desarrollo, no suele aparecer una sola fila.

La siguiente figura representa el intercambio principal; la solicitud posterior del CSS se muestra como una segunda petición independiente:



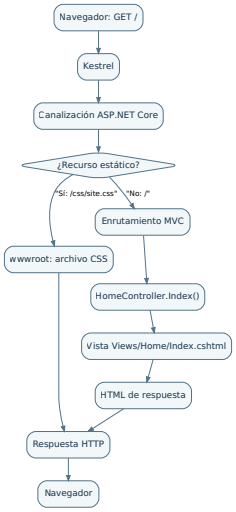
Importante: el navegador **no ejecuta C# ni interpreta código Razor**. Recibe HTML y otros recursos web. Las expresiones Razor se procesan en el servidor al generar la respuesta de esta aplicación MVC. Cuando más adelante estudiemos Blazor, distinguiremos explícitamente su modelo de componentes y su interactividad.

1.1.3. Servidor, aplicación, framework y navegador: cuatro responsabilidades

Conviene separar cuatro participantes que a menudo se confunden:

1. **Kestrel**, el servidor web de ASP.NET Core, escucha en una dirección y puerto y recibe conexiones HTTP.
2. **La canalización de ASP.NET Core**, compuesta por middleware y selección de endpoints, procesa la solicitud y decide qué código puede atenderla.
3. **El código de la aplicación MVC**, con controladores y vistas, selecciona la respuesta y genera su contenido. En nuestra primera aplicación, el controlador `HomeController` devuelve `Views/Home/Index.cshtml`.
4. **El navegador** recibe y representa el documento final. Su trabajo visible no implica que conozca las clases del servidor.

Un middleware es un componente que participa en el procesamiento de una petición y puede atenderla directamente o dejar que continúe. En nuestro `Program.cs`, `UseStaticFiles` permite devolver archivos como la hoja de estilos, mientras que `UseRouting` y el mapeo de ruta convencional intervienen en la selección de la acción MVC. No necesitamos dominar todavía cada middleware: basta con reconocer que el servidor no salta mágicamente de la URL a una función cualquiera.



La figura resume el caso del curso. Una aplicación ASP.NET Core más grande puede utilizar autenticación, sesiones, manejo de excepciones u otros middleware; por ahora no los incorporamos.

1.1.4. Cómo se conecta la URL con nuestro primer controlador

Observemos los archivos reales de `M01_MVC/PROYECTO/1.1/CursoWeb.Mvc`. El arranque está definido en `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Examinemos cada parte, sin adelantar toda la materia de routing del punto 1.4:

`CreateBuilder(args)` prepara un constructor de aplicación con la configuración y los servicios comunes de ASP.NET Core. Aquí no se crea una vista ni se responde todavía a ninguna petición.

`AddControllersWithViews()` registra los servicios que necesitan los controladores MVC y su infraestructura de vistas. El verbo *Add* se refiere a configurar servicios; no significa que se esté invocando una acción concreta. Los servicios se configuran al arrancar la aplicación.

`Build()` construye la aplicación que utilizará dicha configuración.

`UseStaticFiles()` habilita la atención de archivos estáticos ubicados, en nuestro ejemplo, bajo `wwwroot`. Cuando una página hace referencia a `/css/site.css`, ese recurso no necesita pasar por `HomeController`.

`UseRouting()` participa en la identificación del endpoint que corresponde a una petición. La aplicación registra después una ruta convencional mediante `MapControllerRoute`. El patrón `{controller=Home}/{action=Index}/{id?}` dice que, si una URL no especifica controlador ni acción, se usan `Home` e `Index` como valores predeterminados. `id?` representa un segmento opcional que utilizaremos posteriormente.

`Run()` inicia el host y mantiene la aplicación atendiendo peticiones. Es distinto de la ejecución de una acción: el arranque ocurre una vez por proceso; las peticiones pueden atenderse muchas veces.

Ahora observemos el controlador que ya existe:

```
using Microsoft.AspNetCore.Mvc;

namespace CursoWeb.Mvc.Controllers;

public class HomeController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        // Response.Headers["X-Curso-Paso"] = "1.1.4";
        return View();
    }
}
```

La clase deriva de `Controller` para disponer de la infraestructura y de métodos auxiliares de MVC. `[HttpGet]` limita esta acción a solicitudes HTTP GET. La acción `Index` devuelve un `IActionResult`: en este caso, `View()` crea un resultado que posteriormente localizará y representará la vista. **No devuelve una cadena HTML escrita manualmente** ni significa que el navegador pueda ejecutar la vista.

Por convención, `return View();` en `HomeController.Index` selecciona `Views/Home/Index.cshtml`. Esa convención relaciona nombres de controlador, acción y archivo. La estudiaremos con detalle en los puntos 1.3, 1.4 y 1.5.

1.1.5. La vista Razor y el HTML que llega al cliente

El contenido visible de la vista inicial es:

```
@{
    ViewData["Title"] = "Primer recorrido MVC";
}

<h1>1.1 | Primer recorrido MVC</h1>
<p>Esta página HTML ha sido generada en el servidor mediante una acción MVC.</p>
<p id="resultado-inicial">Respuesta HTTP correcta: aplicación MVC disponible.</p>
```

Una vista Razor permite combinar HTML con instrucciones Razor del lado del servidor. En este primer ejemplo, la expresión `ViewData["Title"]` proporciona un título que utiliza el diseño general de la página. No necesitamos todavía dominar el paso de modelos tipados; se abordará en el punto 1.5.

El archivo `Views/Shared/_Layout.cshtml` dispone el esqueleto común del documento: `<html>`, `<head>`, navegación, área principal y pie. Su instrucción `@RenderBody()` inserta el contenido principal de la vista. El archivo `Views/_ViewStart.cshtml` indica que las vistas utilizarán ese layout. Como resultado, la respuesta HTML completa tiene más elementos de los que aparecen directamente en `Index.cshtml`.

¿Por qué no colocamos todos los elementos en una sola vista? Porque compartir un layout evita repetir cabeceras, navegación y estilos en muchas páginas. Por el momento basta con **reconocer** esta composición; en el punto 1.5 aprenderemos a modificar layouts y crear vistas parciales.

La hoja de estilos `wwwroot/css/site.css` se sirve por separado. Aunque la apariencia cambie con el CSS, la acción del controlador no necesita conocer detalles del navegador.

1.1.6. Observar HTTP en lugar de imaginarlo

La explicación del recorrido anterior se puede comprobar de tres maneras que se complementan:

En el navegador, la página visual muestra el contenido procesado. Al abrir las herramientas de desarrollo (F12) y la pestaña *Red/Network*, una navegación permite observar el documento y sus

recursos relacionados. Seleccionando la fila de tipo *document* comprobamos el método GET, la URL y el código 200.

En las cabeceras, la respuesta de un documento HTML contiene `Content-Type: text/html`, posiblemente acompañado de una indicación de codificación. Al pedir el CSS observaríamos en cambio `text/css`. Las cabeceras describen la respuesta y no deben confundirse con los elementos `<h1>` o `<p>`, que forman parte del HTML.

En una herramienta HTTP como `curl`, podemos ver la respuesta sin que intervenga la representación visual del navegador:

```
curl -i http://localhost:5091/
curl -i http://localhost:5091/ruta-inexistente
```

La primera petición debe recibir `200` y una página HTML. La segunda, al no corresponder con una acción de la aplicación, debe recibir `404`. El HTML exacto puede incluir detalles adicionales de framework, pero lo que comprobaremos es el estado HTTP y la marca visible de la vista.

Durante la práctica activaremos dos pequeñas modificaciones y compararemos sus efectos:

- Añadir la cabecera `X-Curso-Paso: 1.1.4` desde la acción MVC **modifica los metadatos HTTP de la respuesta**, no el contenido visual.
- Descomentar un bloque HTML en `Index.cshtml` **modifica el cuerpo HTML**, no necesita añadir una nueva ruta ni cambiar el código HTTP de éxito.

Esta distinción resulta fundamental cuando, más adelante, debamos depurar formularios y llamadas a API: no todos los cambios en el servidor se observan del mismo modo en la pantalla.

1.1.7. MVC y Web API: parecido de infraestructura, diferencia de intención

Comparación orientativa, a partir de una misma necesidad de mostrar datos:

Aspecto	Acción MVC con vista Razor	Endpoint o controlador Web API
Caso más común	Generar una página para una persona	Facilitar datos/operaciones a un consumidor
Respuesta típica	HTML	JSON
Presentación	Se construye en el servidor mediante una vista	El consumidor decide cómo presentar los datos
Infraestructura HTTP	ASP.NET Core y routing	ASP.NET Core y routing
¿Puede acceder a servicios y EF Core?	Sí	Sí
¿Es obligatoria una base de datos?	No	No

Un fragmento conceptual de MVC podría ser `return View(productos);`. Su correspondiente estilo de API podría ser `return Ok(productos);`. En el primer caso se solicita la representación de una vista; en el segundo se envían datos con un resultado HTTP apropiado. Son ejemplos de **intención**: no debemos copiar el segundo en el controlador del proyecto 1.1 porque aún no hemos creado ningún modelo `Producto`.

En particular, no es correcto afirmar que *MVC significa servidor y API significa cliente*: ambos códigos se ejecutan en el servidor. Tampoco es correcto afirmar que MVC no utiliza HTTP porque el usuario solo ve un sitio web: cada navegación sigue siendo una comunicación HTTP.

1.1.8. Confusiones frecuentes y cómo resolverlas

«Si abro localhost en el navegador, estoy abriendo un archivo HTML». En nuestro ejemplo no: el navegador solicita la ruta `/` a un proceso web que escucha en un puerto. La vista se genera como respuesta de una acción MVC. Si apagamos el proceso, esa dirección deja de responder.

«El archivo Index.cshtml se descarga y Razor se ejecuta en el ordenador del alumno». No. Razor se evalúa en el servidor. El resultado enviado al navegador es HTML. La extensión `.cshtml` identifica el archivo fuente del proyecto, no el formato de respuesta entregado al usuario.

«View() devuelve directamente una página HTML». Más exactamente, `View()` crea un resultado MVC de vista. La infraestructura ejecuta ese resultado, localiza la vista y genera la respuesta HTML. La distinción explica por qué la acción puede permanecer compacta.

«Una ruta 404 implica que toda la aplicación ha fallado». No necesariamente. La aplicación puede estar funcionando perfectamente y no tener un endpoint compatible con la URL solicitada. En nuestra primera aplicación es normal que `/ruta-inexistente` devuelva 404.

«Si no veo cambios en la pantalla, el código del controlador no se ha ejecutado». No siempre. Una acción puede añadir una cabecera HTTP que solo se aprecia en la respuesta o en las herramientas de desarrollo.

«ASP.NET Core MVC y ASP.NET Core Web API son frameworks totalmente aislados». Comparten la misma plataforma web y numerosos servicios. Usan convenciones y tipos de respuesta con objetivos diferentes.

1.1.9. Relación directa con la práctica

La práctica `P1.1` reproduce exactamente los hechos descritos aquí: creación de una plantilla MVC, apertura del proyecto inicial del curso, arranque de Kestrel, navegación por `/`, lectura del método y estado de la petición, comparación con una ruta 404, activación de una cabecera y activación de un bloque HTML Razor.

Al finalizar se habrá demostrado un flujo real, no solo memorizado un diagrama. El conocimiento adquirido nos permitirá desarrollar en **1.2** las responsabilidades de Modelo, Vista y Controlador; en **1.3**, la estructura del proyecto; en **1.4**, rutas y acciones; y en **1.5**, vistas Razor más elaboradas.

Lecturas de referencia

- Microsoft Learn, Información general de ASP.NET Core MVC: https://learn.microsoft.com/es-es/aspnet/core/mvc/overview?view=aspnetcore-8.0
- Microsoft Learn, Tutorial de primera aplicación MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/?view=aspnetcore-8.0
- Microsoft Learn, Fundamentos de middleware: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-8.0

1.2. Arquitectura Model-View-Controller

Objetivos

Al finalizar el capítulo, el alumno debe poder identificar sin ambigüedades las responsabilidades de **Model**, **View** y **Controller**; explicar cómo se relacionan durante una petición HTTP; entregar objetos desde una acción a una vista fuertemente tipada; y razonar por qué separar datos, decisión de la respuesta y presentación facilita modificar y probar una aplicación.

En el punto 1.1 observamos una acción `HomeController.Index()` que devolvía una vista. En el 1.2 incorporamos una pieza que aún faltaba: un **modelo de producto**. Usaremos un catálogo con tres objetos en memoria para evitar que el aprendizaje de MVC quede oculto tras una base de datos. El acceso mediante EF Core llegará en M02.

1.2.1. El problema de una aplicación que mezcla todas las responsabilidades

Supongamos que queremos mostrar una página con tres productos. Una solución apresurada consiste en escribir en una misma acción el contenido de los productos y el marcado HTML que aparecerá en pantalla:

```
// Ejemplo conceptual que NO debe copiarse al proyecto.
public IActionResult Index()
{
    string html = "<h1>Productos</h1><p>Cuaderno: 4,50 €</p>";
    return Content(html, "text/html");
}
```

Este código es válido como demostración aislada: la acción puede devolver una respuesta HTTP con contenido HTML. Sin embargo, combina dos decisiones diferentes. Por una parte define qué productos existen; por otra decide cómo pintarlos. Si queremos añadir un producto, cambiar un precio, modificar un elemento `<p>` por una tabla o traducir la presentación, tendremos que editar la misma acción. Conforme crece el código, las responsabilidades se superponen.

¿Y si queremos mostrar esos productos además en una ficha de detalle o exportar un listado? Copiar cadenas HTML empeoraría el problema. ¿Y si queremos comprobar que una regla del dominio es correcta sin abrir un navegador? Si toda la lógica está incrustada en el marcado, la comprobación resulta más difícil.

Este es el problema al que responde la separación de responsabilidades: cada elemento se centra en un tipo de decisión y expone una relación clara con los demás. **Separar no significa crear muchas capas empresariales**: para la dimensión didáctica de nuestro catálogo, tres tipos de archivo son suficientes.

1.2.2. Tres responsabilidades distintas

El patrón arquitectónico **Modelo–Vista–Controlador (MVC)** organiza una aplicación web en tres familias de responsabilidades. «Familia» es una palabra importante: una aplicación real puede tener múltiples modelos, controladores y vistas; no implica una sola clase de cada tipo.

Modelo — representar los datos y las reglas de negocio

Un modelo expresa los conceptos que maneja la aplicación. Para nuestro ejemplo, un producto tiene identificador, nombre y precio.

```
namespace CursoWeb.Mvc.Models;

public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

El objeto `Producto` **no sabe cuál es la URL** donde se mostrará, **no conoce el navegador** ni necesita saber si su representación final será una lista HTML, una tabla o un documento JSON. Es un tipo de C# que recoge información de nuestro dominio. Se ha elegido `decimal` para el precio porque se trata de una magnitud monetaria, y los valores se expresarán en el código con el sufijo `m`.

Que una clase esté en la carpeta `Models` no la convierte automáticamente en una base de datos. En el 1.2 crearemos los objetos en una lista en memoria; se pierden al detener el proceso. En el día 2 reutilizaremos estos conocimientos para persistir productos con EF Core.

En aplicaciones mayores podemos encontrar modelos de dominio, modelos de persistencia y **view models** (tipos adaptados a una vista). No son necesariamente la misma clase. En este ejercicio utilizaremos directamente `Producto` como modelo de la vista porque es pequeño y evita introducir una arquitectura adicional. Más adelante podremos distinguir cuándo convendría proyectar datos a un tipo específico para la presentación.

Vista — representar la información para el usuario

La vista define cómo se transforma la información recibida en una interfaz. En MVC, suele ser un archivo `.cshtml` que combina HTML y sintaxis Razor.

Una vista de productos puede recibir una lista y recorrerla:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?

@if (Model is null)
{
    <p>El controlador no ha enviado una lista.</p>
}
else
{
    <p>Recibidos: @Model.Count productos.</p>
    <ul>
    @foreach (var producto in Model)
    {
        <li>@producto.Nombre</li>
    }
    </ul>
}
```

Este fragmento muestra una vista **tipada**. La directiva `@model` especifica el tipo de datos que espera. La propiedad `Model` permite acceder a los objetos que el controlador le ha proporcionado. Usamos `IReadOnlyList<Producto>?` porque la vista únicamente necesita consultar los datos y durante una etapa inicial el controlador todavía no envía ningún modelo, por lo que puede ser `null`.

El bloque `@if` decide la **presentación** del caso en que no se ha enviado una colección. `@foreach` recorre los productos para crear elementos `<li>`. No introduce productos nuevos, no consulta una base de datos ni toma decisiones empresariales: transforma datos en HTML.

Esto no significa que en una vista esté prohibida toda expresión de C#. Razor permite expresiones y condicionales adecuados para la presentación. Lo que evitaremos es poner en la vista operaciones de persistencia o lógica de negocio compleja.

Controlador — atender la interacción y coordinar la respuesta

El controlador recibe el resultado del enrutamiento, obtiene o prepara los datos necesarios y elige qué respuesta entregar. En nuestro ejercicio, recibe `GET /Productos`, construye tres productos fijos y los pasa a la vista:

```
using CursoWeb.Mvc.Models;
using Microsoft.AspNetCore.Mvc;

namespace CursoWeb.Mvc.Controllers;

public class ProductosController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        var productos = new List<Producto>
        {
            new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
            new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
            new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
        };

        return View(productos);
    }
}
```

La acción es intencionadamente sencilla: no ejecuta consultas ni reglas de negocio complejas. La lista fija nos permite seguir el recorrido sin ruido técnico. En un sistema de producción normalmente delegaría la recuperación de productos en otro servicio, repositorio o mecanismo de acceso a datos, pero esos elementos no son necesarios para comprender este capítulo.

La línea decisiva es `return View(productos);`. Se crea un resultado de vista asociado al modelo que se quiere representar. MVC localiza la vista correspondiente a `ProductosController.Index` y establece su propiedad `Model` con la lista suministrada. En ese momento, la vista ya no necesita inventarse cuáles son los tres productos: los conoce a través de un contrato tipado.

Mapa de responsabilidades

Pregunta	Componente principal
¿Qué atributos tiene un producto?	Modelo `Producto`
¿Qué respuesta se prepara para `GET /Productos`?	`ProductosController.Index`
¿Qué productos se entregan a la pantalla en esta demostración?	El controlador prepara la lista; en un sistema mayor consultaría un servicio
¿Qué HTML representa la cantidad y los nombres?	`Views/Productos/Index.cshtml`
¿Qué parte solicita y representa el HTML?	Navegador, que **no** forma parte de las tres piezas del patrón

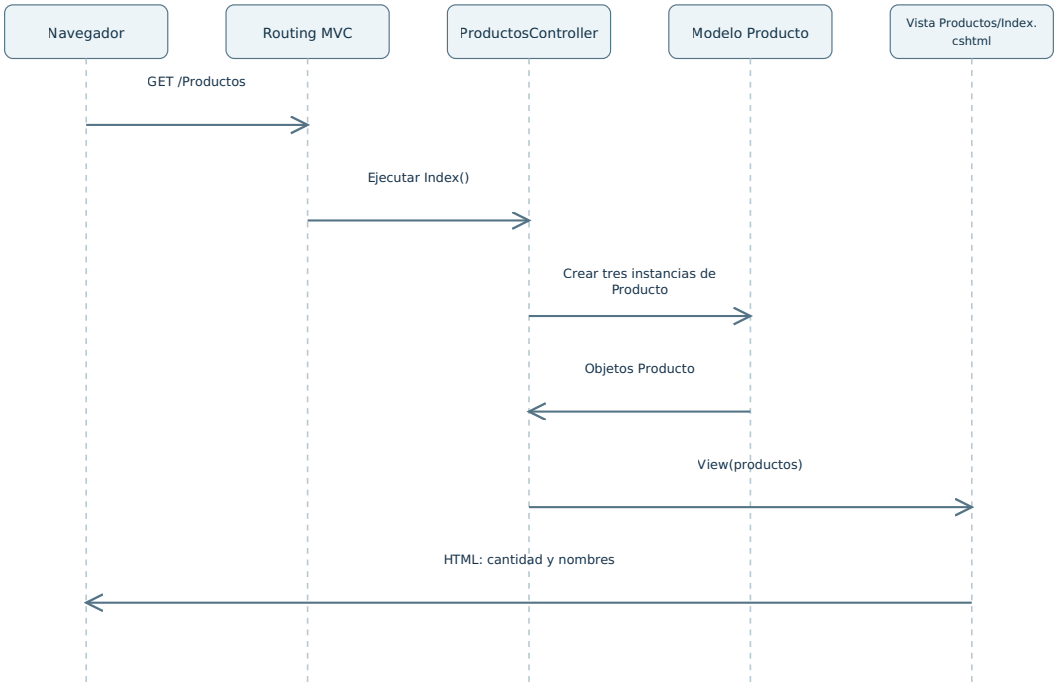
El controlador puede conocer el modelo y seleccionar la vista. La vista suele recibir datos del modelo. **El modelo no necesita depender del controlador ni de la vista**: esa independencia es una de las ventajas más relevantes de MVC.

1.2.3. El recorrido completo de una petición

La separación cobra sentido cuando vemos las tres piezas trabajando juntas. En `Program.cs`, el patrón de ruta convencional registrado es:

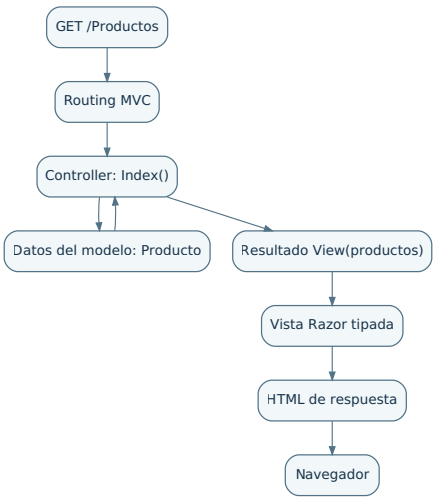
```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Para `GET /Productos`, el segmento «Productos» identifica el controlador y, al omitirse la acción, se utiliza el valor predeterminado «Index». La infraestructura MVC selecciona `ProductosController.Index()`; la acción pasa la lista al resultado de vista; y Razor genera la página que se devuelve al navegador.



Este diagrama representa relaciones **lógicas**: crear objetos del modelo no implica hacer una llamada HTTP a una clase ni que el modelo sea un servicio activo; son instancias C# creadas dentro del mismo proceso. Asimismo, la vista no envía por sí sola un paquete de red al navegador: ASP.NET Core ejecuta el resultado de vista y construye la respuesta HTTP.

El flujo también puede resumirse así:



Es importante entender dos diferencias temporales:

1. **En el arranque**, `Program.cs` registra servicios y rutas. No construye la lista para esta vista.

2. **Ante cada petición**, la acción `Index()` se ejecuta y crea una lista nueva en memoria. En este ejemplo no existe un repositorio persistente: cambiar un producto en el código requiere reconstruir o volver a ejecutar el programa para observarlo.

¿Qué ocurre con `/Productos/Index`? La ruta expresa controlador y acción de forma explícita y, por tanto, llega a la misma acción. Esta es una **convención de rutas** que ampliaremos en el capítulo 1.4; en 1.2 solo necesitamos reconocerla y comprobarla.

1.2.4. Qué significa «pasar un modelo a una vista»

Para distinguir entrega de datos y representación, construiremos el proyecto en estados sucesivos.

Estado A: la acción devuelve `View()` sin datos. La vista existe y puede renderizarse, pero su propiedad `Model` es `null`. Nuestra plantilla muestra «El controlador todavía no ha entregado una lista a la vista». La ruta es válida y puede responder HTTP 200: no recibir datos de modelo no implica necesariamente una respuesta 404.

Estado B: la acción devuelve `View(productos)`. La vista recibe tres objetos y muestra «El controlador ha enviado **3** productos». Todavía no hemos activado el bloque HTML de listado. Esto permite comprobar primero el **traspaso de información** entre controlador y vista sin atribuir el resultado a un bucle visual.

Estado C: se activa `@foreach` en la vista. Entonces aparecen Cuaderno, Lápiz y Carpeta. No hemos cambiado el controlador: la vista simplemente utiliza un modelo que ya recibía. Este cambio ilustra con claridad la separación entre suministrar datos y representarlos.

Estado D: habilitamos un enlace Productos en el layout. El navegador puede ir del inicio al listado. No necesitamos modificar la clase `Producto` para cambiar la navegación; tampoco necesitamos crear un nuevo controlador.

Estado	Controlador	Vista	Navegación
A, base	`View()`	Mensaje sin modelo	Solo Inicio
B, datos	`View(productos)`	Cuenta los tres objetos	Solo Inicio
C, listado	`View(productos)`	Cuenta y muestra los nombres	Solo Inicio
D, completo	`View(productos)`	Cuenta y muestra los nombres	Inicio y Productos

La práctica no requiere escribir cuatro aplicaciones distintas. En `PROYECTO/1.2/CursoWeb.Mvc` están preparados los fragmentos que el alumno activa paso a paso, y una batería E2E reconstruye cada estado sobre una copia temporal.

1.2.5. Ventajas reales de separar responsabilidades

Modificación localizada. Si queremos presentar precios con dos decimales, cambiaremos el marcado de la vista. Si queremos variar la lista fija, editaremos el código que prepara los datos. La clase de producto puede seguir representando las mismas propiedades en ambos casos. Esto reduce el número de archivos afectados por una modificación.

Comprobación independiente. Podemos verificar que `Producto` almacena identificador, nombre y precio como cualquier otro objeto C#. Podemos comprobar que un controlador entrega una lista y observar mediante HTTP el HTML de la vista. En nuestro laboratorio también podemos verificar los estados intermedios con peticiones reales a Kestrel.

Reutilización. El modelo `Producto` podrá utilizarse en una lista, una vista de detalle y, cuando llegue el momento, en un formulario. No necesitamos duplicar la definición de identificador y precio en cada página.

Colaboración y evolución. Una persona puede modificar el aspecto de una vista mientras otra amplía un controlador, siempre que ambas respeten el contrato de datos. Esta separación no elimina toda dependencia: si cambia el nombre o tipo de una propiedad requerida, habrá que actualizar el código que la consume.

Costes y límites. MVC añade convenciones y archivos. Para una página estática pequeña puede parecer más trabajo que escribir un único HTML. Un proyecto mal organizado puede seguir siendo complejo aunque utilice carpetas llamadas Models, Views y Controllers. MVC **ayuda** a separar responsabilidades; no garantiza automáticamente que el diseño sea correcto.

En el capítulo 1.4 ampliaremos las acciones, parámetros y tipos de resultado; en 1.5 aprenderemos a construir vistas con un diseño reutilizable. En esta introducción no añadiremos patrones Repository, Clean Architecture ni otras capas empresariales que no aporta el ejercicio.

1.2.6. Errores conceptuales frecuentes

«El modelo es la base de datos». No. El modelo representa conceptos y datos. Puede estar asociado a persistencia o existir solo en memoria. La clase `Producto` de 1.2 no abre ninguna conexión a una base de datos.

«El controlador debe escribir todo el HTML». No es su responsabilidad principal. Devuelve un resultado de vista y proporciona datos. El archivo `.cshtml` define el marcado de presentación.

«La vista no puede contener ningún C#». Las vistas Razor permiten expresiones, bucles y condicionales de presentación. Lo que evitaremos es añadir lógica empresarial o de acceso a datos dentro de la vista.

«Si la vista recibe `null`, la ruta siempre devuelve 404». No. Una vista bien diseñada puede mostrar un mensaje informativo y responder HTTP 200. Un 404 corresponde a un recurso o endpoint no encontrado, no automáticamente a la ausencia de un objeto.

«El navegador recibe instancias de `Producto`». En esta aplicación MVC, el navegador recibe HTML generado a partir de esas instancias. El objeto C# permanece en el servidor.

«El nombre de la vista es arbitrario». Si se utiliza `return View()` sin especificar un nombre, MVC recurre a convenciones basadas en el controlador y la acción. Por eso nuestro archivo está en `Views/Productos/Index.cshtml`. Más adelante veremos cómo personalizarlo.

«MVC y Razor Pages son lo mismo porque ambos emplean Razor». No. La vista MVC es un elemento de presentación asociado a una acción de controlador. Una Razor Page estructura la gestión de solicitudes alrededor de una página y su PageModel. La comparación detallada será en el capítulo 2.5.

1.2.7. Relación con la práctica P1.2

La práctica construye un recorrido comprobable en cuatro estados, partiendo del resultado de 1.1: inspeccionar la clase `Producto`, navegar a un controlador que aún no envía datos, activar una lista fija y pasarla mediante `View(productos)`, habilitar el `@foreach` de la vista y, finalmente, mostrar un enlace de navegación. Después se modifica el nombre de un producto para constatar que la vista utiliza los valores del modelo en vez de tener nombres escritos directamente en su HTML.

Es necesario **observar cada estado antes de activar el siguiente**, porque el objetivo principal no es acabar cuanto antes una página sino explicar por qué apareció cada elemento: la cantidad depende de la entrega del modelo, el listado depende de la lógica de presentación y el enlace depende del layout.

Al finalizar, debe poder responderse a la pregunta: *«Cuando cambio el nombre de Cuaderno por Bloc, ¿qué archivo modifico y por qué no tengo que reescribir la vista?»*. La respuesta debe enlazar las responsabilidades de Modelo, Controlador y Vista que acabamos de estudiar.

Referencias técnicas

- Microsoft Learn, patrón MVC y separación de responsabilidades: https://learn.microsoft.com/aspnet/core/mvc/overview?view=aspnetcore-8.0
- Microsoft Learn, tutorial de controladores MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/adding-controller?view=aspnetcore-8.0
- Microsoft Learn, sintaxis Razor y vistas tipadas: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor?view=aspnetcore-8.0

1.3. Estructura, convenciones y puesta en marcha de un proyecto ASP.NET Core MVC

Objetivos del capítulo

Este capítulo permitirá al alumno **orientarse dentro de un proyecto MVC real sin depender de memorizar carpetas**. Al terminar podrá identificar el archivo de proyecto, el punto de entrada, los controladores, los modelos, las vistas y los recursos estáticos; relacionar rutas de archivos con rutas URL; entender qué papel desempeña el archivo de configuración; y modificar un recurso web sin confundirlo con una vista Razor.

Partimos de `CursoWeb.Mvc` en su estado completo del punto 1.2: la ruta `/Productos` ya muestra tres productos y dispone de navegación. El ejercicio de 1.3 no vuelve a construir el catálogo, sino que analiza cómo se organiza y se inicia y añade ejemplos mínimos de configuración y archivos estáticos. Reservamos el diseño de rutas más avanzado y las acciones con parámetros para el punto 1.4.

1.3.1. De un proyecto C# a una aplicación web

Un proyecto MVC sigue siendo un **proyecto .NET**: tiene un archivo `.csproj`, compila código C# y genera ensamblados. Su particularidad es que incorpora el SDK web de ASP.NET Core, archivos Razor y una aplicación anfitriona capaz de atender HTTP.

En nuestro proyecto encontramos:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

La primera línea selecciona `Microsoft.NET.Sdk.Web`, que aporta la configuración y las referencias del framework web necesarias para compilar una aplicación ASP.NET Core. `TargetFramework` fija el destino `net8.0`. `Nullable` habilita las anotaciones de nulabilidad, relevantes cuando una vista pueda recibir un modelo nulo; `ImplicitUsings` permite utilizar espacios de nombres comunes sin repetir siempre `using`.

El proyecto de este capítulo no necesita ningún paquete externo adicional para enseñar la estructura. En M02 añadiremos las dependencias de EF Core y SQLite cuando sean pertinentes. Adelantar esas dependencias aquí entorpecería el objetivo didáctico.

El archivo `.csproj` **no es la aplicación ejecutándose**: describe cómo construirla. El arranque real está en `Program.cs` y se ejecuta cuando iniciamos el proyecto. La compilación puede producir `bin/` y `obj/`, carpetas generadas que no deben confundirse con los archivos fuente y que se excluyen de Git.

1.3.2. Mapa real de carpetas: identificar responsabilidades por su ubicación

En la versión correspondiente al punto 1.3, el árbol de mayor interés es el siguiente:

```
CursoWeb.Mvc/
├── CursoWeb.Mvc.csproj
├── Program.cs
├── appsettings.json
```

```
├── Properties/
│   └── launchSettings.json
├── Controllers/
│   ├── HomeController.cs
│   └── ProductosController.cs
├── Models/
│   └── Producto.cs
├── Views/
│   ├── _ViewImports.cshtml
│   ├── _ViewStart.cshtml
│   ├── Home/
│   │   └── Index.cshtml
│   ├── Productos/
│   │   └── Index.cshtml
│   └── Shared/
│       └── _Layout.cshtml
├── wwwroot/
│   ├── css/
│   │   ├── site.css
│   │   └── catalogo.css
│   └── recursos/
│       ├── guia-publica.txt
│       └── aviso-laboratorio.txt
└── DOCUMENTACION/
    └── nota-interna.txt
```

Controllers. Agrupa las clases que atienden acciones MVC. Por convención, `ProductosController` es un controlador y su acción `Index` se relaciona con la URL `/Productos` bajo nuestra ruta convencional. El código recibe la solicitud y elige un resultado. Los detalles de acciones y routing se desarrollarán en 1.4.

Models. Alberga, en este ejemplo, el tipo `Producto`. Se mantiene independiente del navegador y de las vistas. La carpeta facilita encontrar los tipos del dominio, pero no significa que cualquier archivo allí almacenado vaya a ser persistido automáticamente.

Views. Contiene los archivos Razor que MVC utiliza para representar respuestas HTML. La convención `Views/Productos/Index.cshtml` vincula la vista a `ProductosController.Index`, mientras que `Views/Home/Index.cshtml` corresponde a la portada del `HomeController`. La ubicación es significativa para la búsqueda por convención que realiza MVC; no se publica directamente un archivo `.cshtml` para que el navegador lo ejecute.

Views/Shared. Reúne elementos comunes de presentación. `_Layout.cshtml` define el documento HTML envolvente y la navegación, y `@RenderBody()` deja espacio para el cuerpo de la vista seleccionada.

Views/_ViewStart.cshtml. Ejecuta las instrucciones Razor de inicialización compartidas por las vistas de ese árbol. En este caso, establece `Layout = "_Layout";` para utilizar el diseño común.

Views/_ViewImports.cshtml. Permite aplicar directivas e importaciones comunes. En nuestro ejemplo contiene `@using CursoWeb.Mvc` y `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`; esto prepara el uso de Tag Helpers que veremos con mayor detalle en el día 2. Las importaciones no sustituyen a los controladores ni sirven por sí mismas un recurso HTTP.

wwwroot. Es la **raíz web** predeterminada. Contiene recursos que, si la aplicación dispone del middleware correspondiente y el tipo de archivo está admitido, pueden devolverse directamente mediante solicitudes HTTP. Los archivos CSS y de texto de este capítulo ilustran esa diferencia frente a los archivos C# o Razor del proyecto.

DOCUMENTACION. Es una carpeta propia del ejemplo fuera de `wwwroot`. Su archivo de texto se ha incluido deliberadamente para demostrar que no toda ruta física bajo el proyecto se convierte por defecto en una URL pública. No debe interpretarse como un mecanismo de seguridad para proteger secretos: un repositorio público puede permitir descargar sus archivos directamente desde GitHub.

Properties/launchSettings.json. Define perfiles locales para ejecutar y depurar desde herramientas de desarrollo. No sustituye a `appsettings.json` ni determina por sí solo el puerto de todos los entornos

en los que se publique la aplicación. Cuando se usa `dotnet run --no-launch-profile --urls http://localhost:5091`, el argumento de ejecución establece la URL y el perfil local no se aplica.

En proyectos profesionales pueden existir carpetas adicionales, por ejemplo `Services`, `Data`, `DTOs` o `ViewModels`. No son requisitos de MVC para funcionar; responden a necesidades concretas y las introduciremos solo cuando el temario lo necesite.

1.3.3. Program.cs: qué ocurre al arrancar y qué ocurre ante una petición

El código de arranque actual contiene las operaciones esenciales:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

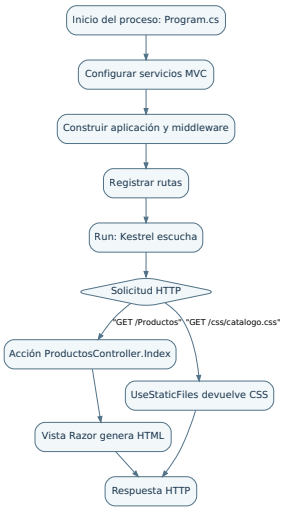
app.Run();
```

La lectura correcta del archivo es **temporal**:

1. `CreateBuilder(args)` prepara configuración, registro de servicios y opciones del host.
2. `AddControllersWithViews()` incorpora los servicios que necesita la infraestructura MVC. No se está mostrando todavía ninguna página.
3. `Build()` construye la aplicación a partir de los servicios configurados.
4. `UseStaticFiles()` incorpora el middleware para atender recursos estáticos reconocidos de `wwwroot`.
5. `UseRouting()` incorpora el middleware de enrutamiento. La ruta convencional registrada mediante `MapControllerRoute` describe los endpoints MVC que la aplicación puede seleccionar.
6. `Run()` inicia el host; a partir de ese momento comienza a atender peticiones en las direcciones configuradas.

Arranque y petición no son lo mismo. Normalmente `Program.cs` se ejecuta una vez al iniciar el proceso, mientras que `ProductosController.Index()` puede ejecutarse tantas veces como un usuario visite `/Productos`. Si se necesitan diez peticiones, no se registran diez veces los servicios. Esta separación ayudará a entender más adelante los ciclos de vida de otros componentes y servicios.

No debemos deducir que `UseRouting()` «dibuja la página» ni que `UseStaticFiles()` sustituye a los controladores. Atienden tipos distintos de trabajo dentro de la plataforma compartida.



El ejemplo utiliza la ruta convencional con controlador y acción predeterminados. Durante 1.3 la identificamos y seguimos; en 1.4 explicaremos sus opciones y su relación con rutas por atributos, parámetros y resultados HTTP.

1.3.4. appsettings.json: configuración distinta del código fuente

Un proyecto web necesita configuraciones que pueden cambiar según dónde y cómo se ejecute. ASP.NET Core carga valores desde fuentes de configuración, entre ellas archivos JSON, variables de entorno y argumentos de ejecución.

Nuestro `appsettings.json` incluye una sección sencilla de propósito didáctico:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "Curso": {
    "Nombre": "Curso MVC de productos",
    "Edicion": "Laboratorio NET8"
  }
}
```

La sección `Logging` controla los niveles de registro; `AllowedHosts` forma parte de la configuración de hosts y no debe entenderse como un sistema completo de protección de solicitudes. La nueva sección `Curso` es propia de nuestra aplicación y no necesita un proveedor externo.

Después de `Build()`, ASP.NET Core permite leer un valor jerárquico con la notación `Curso:Nombre`:

```
app.Logger.LogInformation(
    "P1.3 configuracion: {Nombre}",
    app.Configuration["Curso:Nombre"]);
```

En el proyecto, esta línea aparece inicialmente **comentada** con la marca `PASO 1.3.3`. Cuando se descomenta y se reinicia, la consola muestra el mensaje `P1.3 configuracion: Curso MVC de productos` como registro informativo. La aplicación ya tiene los servicios de logging del host, por lo que no necesitamos instalar otra biblioteca.

El texto `{Nombre}` de la plantilla de logging es un marcador de valor; permite registrar propiedades de forma estructurada. No debe confundirse con las expresiones Razor de una vista ni con los segmentos del patrón de rutas MVC: son sintaxis parecidas en tecnologías diferentes.

Una observación importante: el argumento `--urls` utilizado al arrancar determina las direcciones del servidor en este laboratorio; no estamos cambiando la sección `Curso` para controlar la URL. Aprenderemos en M05 las diferencias elementales entre ejecutar y publicar.

Seguridad básica. Los archivos de configuración del proyecto pueden contener ajustes, pero **no se deben subir secretos reales** a un repositorio público. Este curso solo usa valores didácticos. El hecho de que un archivo esté fuera de `wwwroot` no impide que un usuario con acceso al repositorio GitHub pueda leerlo.

1.3.5. wwwroot: rutas físicas y URLs no son lo mismo

El archivo `wwwroot/css/catalogo.css` está ubicado físicamente bajo la raíz web del proyecto. Su URL por defecto es `/css/catalogo.css`, sin incluir el nombre de carpeta `wwwroot`. Esto ocurre porque `UseStaticFiles()` expone los recursos desde esa raíz.

En la aplicación preparamos este CSS:

```css
/* PASO 1.3.4: recurso estático independiente de los controladores. */
main.container {
    border-left: 5px solid #2d635d;
    padding-left: 1.5rem;
}
```

Mientras el archivo exista bajo `wwwroot`, una solicitud directa a `http://localhost:5091/css/catalogo.css` puede obtenerlo, aunque ninguna vista incluya todavía una referencia a él. La aplicación lo sirve como recurso estático reconocido.

Para que el navegador lo aplique a las páginas MVC, añadimos en `Views/Shared/_Layout.cshtml` una etiqueta HTML:

```html
<link rel="stylesheet" href="/css/catalogo.css" />
```

Al activarla, la siguiente navegación solicita el HTML y, a continuación, el CSS adicional. El estilo añade un borde al área principal y permite comprobar visualmente el efecto. El archivo CSS no se transforma en una vista Razor ni hace falta crear un `CatalogoController` que lo devuelva.

La distinción queda reflejada en esta tabla:

Ubicación física	URL de ejemplo	Resultado esperado
`wwwroot/css/site.css`	`/css/site.css`	HTTP 200, CSS
`wwwroot/css/catalogo.css`	`/css/catalogo.css`	HTTP 200, CSS, incluso antes de enlazarlo
`wwwroot/recursos/guia-publica.txt`	`/recursos/guia-publica.txt`	HTTP 200, texto
`wwwroot/recursos/aviso-laboratorio.txt`	`/recursos/aviso-laboratorio.txt`	HTTP 200, texto
`DOCUMENTACION/nota-interna.txt`	`/DOCUMENTACION/nota-interna.txt`	HTTP 404 con esta configuración

Ubicación física	URL de ejemplo	Resultado esperado
`Views/Productos/Index.cshtml`	`/Views/Productos/Index.cshtml`	No es un documento estático servido por `UseStaticFiles`
`Views/Productos/Index.cshtml`	`/Productos`	El HTML se genera al ejecutar la acción MVC

Este comportamiento corresponde a **la configuración concreta de nuestro proyecto**. ASP.NET Core permite modificar las opciones de los proveedores de archivos y habilitar otras rutas estáticas, pero esa ampliación no entra en el alcance de 1.3.

Un archivo fuera de `wwwroot` puede seguir siendo accesible de otra manera si desarrollamos una acción que lo envíe, configuramos otro proveedor de archivos o lo publicamos mediante una infraestructura externa. Por eso la conclusión correcta no es «fuera de wwwroot significa secreto», sino «`UseStaticFiles()` no lo expone por defecto».

Además, retirar un enlace HTML **no despublica** el archivo que ya está en `wwwroot`. Un usuario que conozca o adivine su URL puede solicitarlo directamente. Esta distinción será esencial en el reto práctico.

1.3.6. Layout, vistas y Tag Helpers: convenciones de presentación

La página `/Productos` se compone de la vista `Views/Productos/Index.cshtml` y del layout compartido `Views/Shared/_Layout.cshtml`. El archivo `_ViewStart.cshtml` establece ese layout; `_ViewImports.cshtml` define importaciones y habilita Tag Helpers para las vistas.

El layout contiene una zona para las hojas de estilo, navegación y `@RenderBody()`. En nuestro ejercicio activaremos dos enlaces comentados:

```
<link rel="stylesheet" href="/css/catalogo.css" />
<a href="/recursos/guia-publica.txt">Guía pública</a>
```

Se trata de HTML con rutas de recursos estáticos. La vista `Index.cshtml` no necesita saber dónde está la hoja CSS ni incorporar cada enlace de navegación; es una decisión de diseño compartida.

En el día 2 estudiaremos Tag Helpers como `asp-for` y `asp-action`. De momento conviene conocer su lugar de registro, pero no anticipar su sintaxis en formularios. Del mismo modo, las vistas tipadas que ya utilizamos en `1.2` se profundizarán en `1.5`.

Un error frecuente consiste en pensar que una carpeta llamada `Views` se distribuye al navegador como si fuera un directorio de archivos web. No es así en nuestro proyecto: son archivos fuente para generar una respuesta **del lado del servidor**. Las hojas CSS, en cambio, sí se entregan como recursos separados al navegador.

1.3.7. Convenciones, ventajas y límites

Las convenciones de MVC reducen configuración repetitiva. Los nombres `ProductosController`, `Index` y `Views/Productos/Index.cshtml` bastan en nuestro ejercicio para conectar acción y vista. La carpeta `Shared` permite compartir el layout. `wwwroot` evita escribir una acción para cada CSS.

No hay que convertir esas convenciones en leyes universales. Una aplicación puede personalizar rutas, nombres de vistas y proveedores de contenido estático. Para avanzar con seguridad, primero aprenderemos el funcionamiento convencional; después abordaremos las personalizaciones que el temario sí necesita.

La organización del proyecto también favorece el aprendizaje incremental: añadir un modelo de producto no modifica necesariamente el layout, y cambiar el CSS no requiere modificar la clase del dominio. Esa es una continuación directa del principio de responsabilidades MVC aprendido en el capítulo 1.2.

Qué no haremos todavía: estructurar una solución de múltiples proyectos, configurar contenedores de dependencias personalizados, introducir infraestructura de autenticación, publicar en IIS/Docker, aplicar bundling complejo ni estudiar todos los orígenes de configuración del host. Esas decisiones no ayudan al objetivo limitado de este capítulo.

1.3.8. El reto de distinguir archivos públicos y archivos del proyecto

La práctica termina con una comprobación sencilla pero reveladora: habilitar en el layout el enlace comentado **Aviso de laboratorio**, que apunta a un archivo ubicado en `wwwroot/recursos`. Después se solicitará por URL otro archivo de texto, `DOCUMENTACION/nota-interna.txt`, situado fuera de `wwwroot`.

El primero debe responder HTTP 200 y el segundo 404 en nuestro proyecto. Más interesante aún, el primero ya era accesible **antes** de habilitar su enlace de navegación. Esta observación muestra que la accesibilidad de un recurso depende de la configuración de publicación, no de que exista un enlace visible en pantalla.

El código completo, las instrucciones para activar el enlace, los resultados esperados y las pruebas automáticas de ambas respuestas están en `P1.3`. El reto no requiere conocimientos de seguridad avanzados ni recursos externos.

1.3.9. Problemas frecuentes

«Una vista Razor se abre por su ruta de archivo». En el flujo MVC habitual se solicita una URL asociada a una acción; la infraestructura localiza la vista y genera HTML. La ruta física del archivo no se equipara a una URL pública.

«Para servir un CSS tengo que crear un controlador». No en esta aplicación: `UseStaticFiles()` y `wwwroot` ya cubren esa necesidad.

«Si no hay enlace, el recurso está oculto». No. Los archivos estáticos pueden solicitarse directamente si se conoce la URL y están publicados.

«Todo el contenido del proyecto está disponible en el navegador». No por defecto. El código C#, los archivos de construcción y otros recursos del proyecto no se sirven automáticamente como estáticos por esta configuración.

«Program.cs se ejecuta por cada petición». No. Configura y arranca el proceso una vez; los controladores atienden las peticiones que llegan después.

«El perfil de Visual Studio y appsettings.json son idénticos». No. `launchSettings.json` describe perfiles de lanzamiento local; `appsettings.json` aporta configuración de aplicación y es solo uno de los orígenes posibles.

«El archivo de configuración incluye valores seguros porque no está en wwwroot». Esto no justifica guardar contraseñas en un repositorio. Los recursos y las políticas de despliegue deben distinguirse del control de acceso al código fuente.

1.3.10. Preparación para la práctica y el punto 1.4

La práctica `P1.3` parte de una página `/Productos` ya funcional, sigue el árbol de archivos, activa un registro de configuración, incorpora una hoja CSS, habilita un enlace a un documento estático y resuelve un reto sobre publicación de recursos.

El alumno debe terminar explicando **en qué carpeta se modifica una cosa y por qué**, además de haber comprobado cada resultado. Esta capacidad permite avanzar al punto `1.4`, donde los controladores atenderán nuevas acciones, parámetros y rutas.

Referencias de Microsoft Learn

- Estructura básica de un proyecto MVC: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-8.0
- Archivos estáticos en ASP.NET Core: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/static-files?view=aspnetcore-8.0
- Configuración en ASP.NET Core: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/?view=aspnetcore-8.0
- Vistas Razor y convenciones: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

1.4. Controladores, acciones, parámetros y routing en ASP.NET Core MVC

Objetivos de aprendizaje

Al finalizar este punto, el alumno sabrá **seguir una URL hasta su acción MVC**, añadir acciones nuevas a un controlador, distinguir una respuesta HTML de una respuesta de texto o JSON, utilizar parámetros provenientes de la URL y controlar los resultados HTTP de éxito y error. También comprenderá **el enrutamiento convencional frente al enrutamiento por atributos**, aplicando ambos al catálogo de productos.

Se trabaja sobre el proyecto 1.4, continuación funcional del 1.3. El listado de productos y su navegación ya funcionan. Construiremos una acción de estado, un detalle tipado y otra dirección para la misma clase de información. El reto final devolverá una respuesta JSON breve desde un controlador MVC para conectar con los conocimientos previos de Web API.

1.4.1. Qué es una acción MVC y cómo se selecciona

En los ejemplos anteriores, `ProductosController` tenía una acción `Index()` que recibía una petición GET y devolvía `View(productos)`. Cuando abrimos `/Productos`, ASP.NET Core no busca un archivo `Productos.html` en el disco. El sistema de enrutamiento identifica un **endpoint** MVC y ejecuta la acción correspondiente.

Una acción es un método público del controlador que puede preparar datos, decidir qué resultado devolver y recibir valores de la petición. Derivar de `Controller` nos proporciona métodos auxiliares como `View()`, `Content()`, `NotFound()`, `BadRequest()` y `Ok()`. No todas las acciones deben devolver una vista, aunque muchas de las páginas MVC del curso lo harán.

El arranque contiene nuestra ruta convencional:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Su plantilla se descompone en tres segmentos:

- `{controller=Home}`: el controlador es Home cuando no se proporciona otro.
- `{action=Index}`: la acción es Index cuando la URL no indica otra.
- `{id?}`: se admite un tercer segmento opcional, que más adelante enlazaremos al parámetro `id` del método.

Una URL como `/Productos/Detalle/2` permite obtener los valores de ruta: controlador = Productos, acción = Detalle, id = 2. La tabla muestra la lectura que haremos:

URL	Controlador	Acción	Valor de id
`/`	Home	Index	Sin valor
`/Productos`	Productos	Index	Sin valor
`/Productos/Index`	Productos	Index	Sin valor
`/Productos/Estado`	Productos	Estado	Sin valor

URL	Controlador	Acción	Valor de id
`/Productos/Detalle/2`	Productos	Detalle	2
`/Productos/Detalle/99`	Productos	Detalle	99

Una **coincidencia de ruta** no garantiza que la acción devuelva 200. Puede seleccionar la acción correctamente, pero esta determinar que el producto no existe y devolver 404. Por eso distinguiremos entre «no hay endpoint que coincida» y «el endpoint se ejecuta y responde con un resultado de error».

Reutilizar la lista de productos sin base de datos

El proyecto 1.3 construía la lista directamente dentro de `Index()`. Para añadir una acción `Detalle(id)` sin escribir una segunda lista distinta, la versión 1.4 extrae los mismos datos a un método privado:

```
private static IReadOnlyList<Producto> ObtenerProductosEjemplo() =>
    new List<Producto>
    {
        new() { Id = 1, Nombre = "Cuaderno", Precio = 4.50m },
        new() { Id = 2, Nombre = "Lápiz", Precio = 1.20m },
        new() { Id = 3, Nombre = "Carpeta", Precio = 3.75m }
    };

[HttpGet]
public IActionResult Index()
{
    return View(ObtenerProductosEjemplo());
}
```

La lista **no es persistente**, y el método la crea en memoria cada vez que se solicita. Usamos esta refactorización mínima exclusivamente para compartir los mismos datos entre el listado, la ficha y el reto. El método privado **no es una acción pública**; no se publica automáticamente como una URL. En M02 sustituiremos los datos fijos por la persistencia con EF Core.

1.4.2. Restricciones de verbo HTTP y respuestas distintas de View

El atributo `[HttpGet]` indica que una acción está destinada a manejar solicitudes GET. En este ejercicio lo utilizamos sin una plantilla de URL propia para permitir que la acción `Estado()` se alcance por la ruta convencional:

```
[HttpGet]
public IActionResult Estado()
{
    return Content("Catalogo MVC disponible", "text/plain");
}
```

Una solicitud a `GET /Productos/Estado` devolverá **HTTP 200**, contenido de tipo texto y la cadena `Catalogo MVC disponible`. No hay vista `Estado.cshtml`: `Content()` crea un tipo de resultado HTTP distinto de `View()`.

Esto demuestra algo esencial para quien ya conoce Web API: **un controlador MVC no está obligado a devolver HTML**. La acción elige un resultado apropiado para la necesidad de la interfaz. Al mismo tiempo, una aplicación que principalmente muestra vistas no tiene por qué convertirse por ello en una API REST completa.

En el punto 2.1 utilizaremos formularios GET/POST. Aquí no introducimos métodos para alterar datos; solo recibimos consultas GET y comparamos sus respuestas.

1.4.3. Parámetros de una acción: el detalle de producto

Vamos a introducir una acción `Detalle(int id)`. La ruta convencional puede capturar el tercer segmento de una URL y la infraestructura de **model binding** convertirlo a un parámetro `int` del método. El binding toma valores de distintas partes de la petición según el caso; durante este capítulo nos centraremos en el valor de ruta `id`.

El código que activaremos en la práctica es:

```
[HttpGet]
public IActionResult Detalle(int id)
{
    if (id <= 0)
    {
        return BadRequest("El identificador debe ser mayor que cero.");
    }

    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);

    if (producto is null)
    {
        return NotFound();
    }

    return View(producto);
}
```

`int id` declara un parámetro numérico. Cuando la solicitud es `GET /Productos/Detalle/2`, recibimos `id = 2`. La acción busca ese identificador dentro de la lista creada por `ObtenerProductosEjemplo()`.

`BadRequest(...)` crea una respuesta de error **HTTP 400** cuando nuestro propio método detecta un valor menor o igual que cero. En este ejemplo, si visitamos `/Productos/Detalle` sin proporcionar id, el parámetro numérico no anulable tiene el valor predeterminado 0 en ausencia del segmento y entra en esa comprobación. No es una regla universal para todos los controladores: depende de la firma y del binding utilizados.

`FirstOrDefault(...)` selecciona un elemento de la lista o devuelve `null` si no existe. Si visitamos `/Productos/Detalle/99`, no existe un producto con id 99 y el código devuelve **HTTP 404** mediante `NotFound()`. Este 404 ocurre **dentro de una acción válida**, porque la ruta y el método sí se encontraron.

`View(producto)` crea un resultado de vista que recibe un objeto `Producto` y produce una respuesta **HTTP 200 con HTML**. Por convención, `ProductosController.Detalle` selecciona `Views/Productos/Detalle.cshtml`.

La vista será fuertemente tipada:

```
@model CursoWeb.Mvc.Models.Producto

<h1 id="titulo-detalle">1.4 | Detalle de producto</h1>
<dl id="detalle-producto">
    <dt>Identificador</dt><dd id="detalle-id">@Model.Id</dd>
    <dt>Nombre</dt><dd id="detalle-nombre">@Model.Nombre</dd>
    <dt>Precio</dt><dd id="detalle-precio">@Model.Precio.ToString("0.00") €</dd>
</dl>
<p><a href="/Productos">Volver al catálogo</a></p>
```

Este ejemplo utiliza `@Model` porque cada solicitud de detalle recibe **un objeto**, no la colección que recibía la vista de `Index`. El listado tiene `@model IReadOnlyList<Producto>?`; la ficha tiene `@model Producto`. Confundir los dos tipos provocaría errores al ejecutar la vista.

La función de `<dl>`, `<dt>` y `<dd>` es representar relaciones nombre/valor en HTML, como identificador, nombre y precio. No cambian el tipo de resultado HTTP. Los atributos `id` del HTML permiten identificar los valores en el navegador y en las pruebas.

1.4.4. IActionResult, resultados concretos y códigos de estado

En una acción hemos utilizado varios métodos de retorno. Es importante no tratar `return View(...)`, `return Content(...)` y `return NotFound()` como si todos significaran «mostrar una página».

Instrucción del controlador	Resultado habitual en este capítulo	Qué recibe el cliente
`return View(producto);`	200, `text/html`	HTML generado desde Razor
`return Content("...", "text/plain");`	200, `text/plain`	Texto sin layout
`return BadRequest("...");`	400	Resultado de error con información explicativa
`return NotFound();`	404	Respuesta de recurso no encontrado
`return Ok(new { ... });`	200, formato JSON negociado habitualmente	Datos JSON del objeto anónimo

El tipo declarado `IActionResult` permite devolver diferentes tipos concretos de resultado desde una misma acción. Una firma `ActionResult<T>` es frecuente en Web API cuando se desea describir un tipo de dato de éxito y posibles errores; aquí utilizamos `IActionResult` para no desviar la práctica hacia el diseño de API.

404: dos situaciones que conviene distinguir

Si `GET /no-existe` no coincide con ninguna ruta de nuestra aplicación, el sistema de enrutamiento termina sin ejecutar una acción de catálogo. Si `GET /Productos/Detalle/99` coincide con `Detalle(int id)`, **la acción sí se ejecuta**, intenta buscar el producto y decide `NotFound()`. Ambas situaciones pueden producir 404, pero sus causas y soluciones no son iguales.

400 frente a 404

Un identificador que nuestra acción considera inválido, como 0, obtiene 400 por una regla explícita del código. Un identificador válido en formato pero inexistente, como 99, obtiene 404. El código de estado **no aparece automáticamente por el hecho de usar un int**: lo decidimos deliberadamente en estos métodos. Verificaremos la diferencia mediante HTTP.

1.4.5. Enrutamiento convencional frente a rutas por atributos

Hasta ahora el registro de `MapControllerRoute` determina el patrón general. Cuando necesitamos una dirección que no respeta `/Productos/NombreAccion/id`, podemos definir **una plantilla de ruta sobre una acción**.

Añadiremos:

```
[HttpGet("/catalogo/ficha/{id:int}")]
public IActionResult Ficha(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
```

```
    {
        return NotFound();
    }

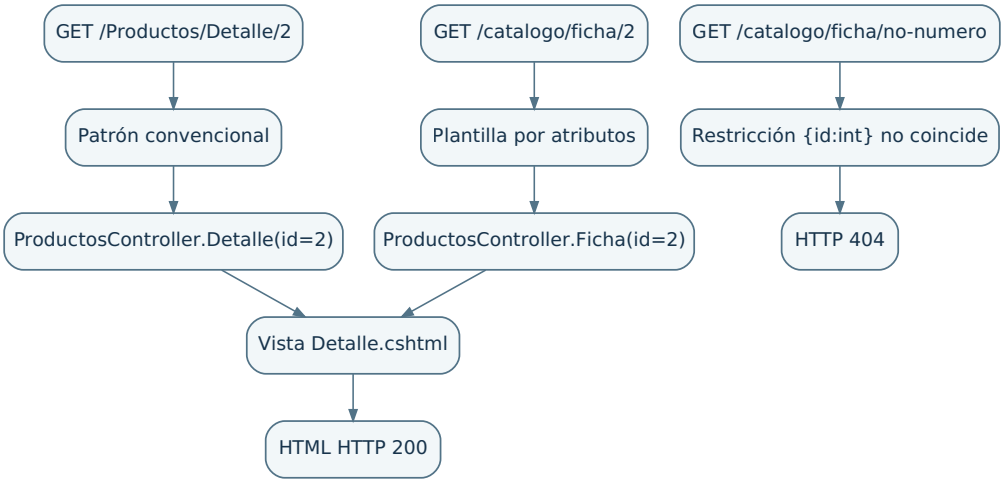
    return View("Detalle", producto);
}
```

La plantilla absoluta `"/catalogo/ficha/{id:int}"` indica una dirección nueva, por ejemplo `GET /catalogo/ficha/2`, aunque el controlador se llame `ProductosController`. El prefijo `/` señala una plantilla absoluta. `{id:int}` exige que el segmento de la ruta pueda interpretarse como entero; una petición `/catalogo/ficha/no-numero` **no cumple la restricción** y no alcanza esta acción, por lo que obtendrá 404.

La acción `Ficha` utiliza la misma colección y retorna `View("Detalle", producto)` para **reutilizar explícitamente** la vista de detalle. Si utilizáramos `return View(producto);` sin especificar nombre, MVC intentaría localizar la vista de nombre `Ficha` por convención. Esta diferencia entre nombre de acción y nombre de vista es uno de los objetivos principales del ejemplo.

Con rutas por atributos, una acción seleccionada exclusivamente por su plantilla no adquiere necesariamente una URL convencional equivalente. En nuestro caso, `/Productos/Ficha/2` no corresponde al endpoint definido; la dirección prevista es `/catalogo/ficha/2`.

Los dos enfoques pueden coexistir en una aplicación. **No significa que una ruta por atributos sea siempre mejor** ni que debamos reemplazar el patrón convencional para todas las páginas. Lo importante es saber qué URL se publica en cada caso y qué restricción acepta.



1.4.6. Enlaces de navegación desde la vista del listado

Una ficha de detalle resulta mucho más útil cuando se puede abrir desde el listado. La vista de `Index` ya recorre la colección `Model`. Dentro de cada elemento de la lista añadiremos:

```
<a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
```

Razor inserta el valor de `producto.Id` en el atributo `href`. Para los tres objetos, las URLs serán `/Productos/Detalle/1`, `/Productos/Detalle/2` y `/Productos/Detalle/3`. Cada enlace usa la misma acción pero envía un **valor de parámetro diferente**.

El código está comentado en el proyecto como `PASO 1.4.5` para que el alumno lo active después de crear y verificar la acción `Detalle`. Activar el enlace antes no impide que el listado cargue, pero conduciría a una URL todavía sin acción y provocaría una navegación fallida. La secuencia docente evita ese problema.

En este punto escribimos el atributo `href` directamente para visualizar qué URL se forma. En M02 aprenderemos los Tag Helpers que permiten generar enlaces y formularios usando metadata MVC en lugar de escribir siempre las rutas a mano.

1.4.7. Reto: devolver JSON desde un controlador MVC

El reto conecta las distintas ideas del capítulo: crear una acción con parámetro, publicarla mediante una plantilla de atributo, seleccionar un producto y decidir entre un resultado **200 JSON** y un **404**.

El método, incluido comentado en el proyecto, es:

```
[HttpGet("/catalogo/resumen/{id:int}")]
public IActionResult Resumen(int id)
{
    var producto = ObtenerProductosEjemplo().FirstOrDefault(p => p.Id == id);
    if (producto is null)
    {
        return NotFound();
    }

    return Ok(new { producto.Id, producto.Nombre });
}
```

La URL `/catalogo/resumen/2` selecciona el producto Lápiz y produce un objeto JSON semejante a:

```
{"id":2,"nombre":"Lápiz"}
```

La serialización JSON habitual de ASP.NET Core utiliza nombres de propiedades en `camelCase` para este resultado. No ejecuta una vista Razor ni modifica la plantilla de datos. Una consulta `/catalogo/resumen/99` devuelve 404 porque el método no encuentra un producto. Si se solicita un segmento no entero, la ruta con `{id:int}` no coincide y también se obtiene 404, en ese caso sin ejecutar la acción.

Para un desarrollador que conoce Web API, esto confirma que **MVC comparte los tipos fundamentales de resultado HTTP**, aunque nuestro curso utilice principalmente vistas Razor para representar la interfaz del usuario. En M04 construiremos una API auxiliar mínima que será consumida desde Blazor; el propósito de este reto es observar formatos de respuesta, no enseñar toda la disciplina de diseño de API.

1.4.8. Errores habituales y diagnóstico

«La ruta existe, luego siempre responde 200». Falso. `Detalle(99)` y `Detalle(0)` coinciden con una acción pero responden 404 y 400, respectivamente, por decisiones expresas del controlador.

«Tengo que crear una vista Estado.cshtml». No, porque la acción usa `Content()`. Solo necesitamos una vista si elegimos un resultado de vista.

«La ruta Ficha se construye como /Productos/Ficha/2». No en este ejemplo. El atributo `[HttpGet("/catalogo/ficha/{id:int}")]` define otra URL, que no coincide con el patrón convencional habitual de esa acción.

«No entiendo por qué la ficha Ficha utiliza Detalle.cshtml». Porque la instrucción `View("Detalle", producto)` señala explícitamente el nombre de la vista compartida por ambas acciones.

«La acción no recibe el id». Revisar que la ruta tiene el tercer segmento y que la firma recibe `int id`. En el caso del atributo hay que escribir `{id:int}` y llamar igual al parámetro que la plantilla vincula.

«Al activar el comentario, aparece un error de compilación». Comprobar que se han quitado **los dos** delimitadores del bloque y no se han alterado las llaves, la indentación semántica ni las llamadas a métodos.

«Una respuesta 404 siempre significa que el servidor está apagado». No. Un servidor apagado provoca normalmente un error de conexión. Un 404 es una respuesta HTTP que el servidor ha generado.

«El navegador recibe un objeto Producto cuando abre el detalle». No directamente: `View(producto)` usa el objeto en el servidor para producir HTML. En el reto `Ok(new { ... })` sí se serializa un objeto a JSON.

1.4.9. Relación con P1.4 y continuidad

La práctica comienza inspeccionando el controlador existente y la lista compartida; activa `Estado()` y observa texto plano; activa `Detalle(int id)` y comprueba 200/400/404; incorpora una ruta por atributos; añade enlaces por producto en la vista; y realiza el reto JSON, igualmente comentado en el proyecto.

La batería de pruebas compila y ejecuta **cada estado solicitado** y verifica sus respuestas reales a través de HTTP, sin reemplazar la comprensión manual de la navegación. Se preservan también las pruebas de regresión de los capítulos 1.1 a 1.3.

El capítulo siguiente, **1.5**, profundizará en la sintaxis Razor, la composición de vistas, los layouts y las vistas parciales usando las páginas y el detalle funcional de este capítulo.

Fuentes y referencias

- Microsoft Learn, enrutamiento a acciones de controlador: https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing?view=aspnetcore-8.0
- Microsoft Learn, solicitudes mediante controladores: https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/actions?view=aspnetcore-8.0
- Microsoft Learn, resultados de acciones de controlador: https://learn.microsoft.com/en-us/aspnet/core/web-api/action-return-types?view=aspnetcore-8.0
- Microsoft Learn, vistas Razor y tipado de modelos: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

1.5. Vistas Razor, modelos tipados, layouts y vistas parciales

Objetivos y contexto

En el punto 1.4 construimos una acción de detalle de producto, comprobamos distintos resultados HTTP y añadimos enlaces desde el listado. Ahora el foco se traslada a **cómo se produce y se organiza el HTML de una aplicación ASP.NET Core MVC**.

Al terminar, el alumno podrá interpretar las directivas y expresiones básicas de Razor, comparar una vista de colección con una vista de objeto individual, utilizar un layout común, reconocer una sección opcional, incorporar una vista parcial tipada y reutilizarla desde dos vistas diferentes. También podrá explicar por qué la misma regla de presentación cambia el resultado de distintas páginas sin alterar los controladores.

No adelantaremos los formularios y los Tag Helpers de entrada de datos de M02. El dominio sigue siendo un pequeño catálogo de tres productos y no necesita una base de datos.

1.5.1. Qué es Razor y dónde se ejecuta

Razor es la sintaxis de plantillas de ASP.NET Core que combina HTML con expresiones y bloques de C#. En MVC, la infraestructura de vistas procesa los archivos `.cshtml` **en el servidor**, usando el modelo que la acción entrega, y genera el HTML de la respuesta HTTP.

No es un lenguaje que el navegador interprete. Cuando el navegador muestra «Lápiz», no está ejecutando `@Model.Nombre`: esa expresión ya se ha evaluado en el servidor. En las herramientas de desarrollo veremos el resultado HTML, no el archivo Razor original como código ejecutable.

Ejemplos básicos:

```
@model CursoWeb.Mvc.Models.Producto

<h2>@Model.Nombre</h2>
<p>Precio: @Model.Precio.ToString("0.00") €</p>
```

La primera línea es una **directiva Razor** y no genera por sí misma un elemento HTML. Declara que la vista recibirá un objeto `Producto` como modelo. Las expresiones `@Model.Nombre` y `@Model.Precio.ToString("0.00")` insertan valores en el marcado. El resultado de una petición de detalle con `id=2` contiene un encabezado con el nombre del producto y un párrafo con el precio.

El símbolo `@` tiene diferentes usos según el contexto. Es importante diferenciarlos antes de comenzar a copiar ejemplos:

Forma	Qué representa	Ejemplo
`@model`	Directiva que declara el tipo de modelo	`@model Producto`
`@{ ... }`	Bloque de código C#	`@{ ViewData["Title"] = "Catálogo"; }`
`@Model.Nombre`	Expresión cuyo valor se escribe en HTML	`<h3>@Model.Nombre</h3>`
`@if (...) { ... }`	Condición Razor que decide qué fragmento HTML se genera	Mensaje de catálogo amplio

Forma	Qué representa	Ejemplo
`@foreach (...) { ... }`	Iteración que puede generar elementos HTML repetidos	Tarjetas de productos
`@* ... *@`	Comentario Razor que no se representa en el HTML final	Bloques que activamos en prácticas

Como en cualquier plantilla, **la estructura importa**. El C# dentro de `@if` o `@foreach` puede utilizarse para controlar el marcado generado, pero no conviene trasladar a las vistas las consultas a bases de datos o las reglas empresariales del sistema. Las vistas deben centrarse en la presentación.

HTML codificado de forma segura

Cuando Razor genera contenido desde una expresión ordinaria, codifica caracteres que podrían interpretarse como HTML. Esta protección es relevante para mostrar nombres escritos por usuarios, aunque todavía utilicemos datos fijos. Evitaremos utilizar `Html.Raw` sobre texto no confiable. El hecho de que en la respuesta aparezca una entidad HTML para una letra acentuada no implica que se haya perdido el texto: el navegador la representa correctamente.

1.5.2. Una vista de lista y una vista de detalle no reciben el mismo modelo

El proyecto utiliza dos vistas tipadas. La vista del catálogo declara:

```
@model IReadOnlyList<CursoWeb.Mvc.Models.Producto>?
```

Esto indica que espera **una colección de productos** y que el modelo podría ser nulo. Su estructura actual comienza con una condición para ese caso y, cuando hay lista, muestra la cantidad y los productos:

```
@if (Model is null)
{
    <p id="sin-modelo">El controlador no ha proporcionado productos.</p>
}
else
{
    <p id="numero-productos">
        El controlador ha enviado <strong>@Model.Count</strong> productos.
    </p>
}
```

La página de detalle, por su parte, declara:

```
@model CursoWeb.Mvc.Models.Producto
```

y consulta `@Model.Id`, `@Model.Nombre` y `@Model.Precio`. Para una acción `Detalle(int id)` que entrega un único objeto, la vista no necesita un `foreach` sobre una colección.

No debemos intercambiar los contratos: una vista que declara `Producto` no puede tratar automáticamente su modelo como una lista. El contrato tipado facilita comprender el código y detecta errores de acceso a propiedades. Una vista también puede utilizar un *view model* distinto del tipo de dominio; por ahora emplearemos `Producto` directamente, porque la práctica se concentra en la sintaxis Razor y no en arquitecturas de capas.

Una condición de presentación adicional

En el proyecto 1.5 existe un bloque comentado del listado:

```
@if (Model.Count >= 3)
{
    <p id="catalogo-completo">El catálogo contiene al menos tres productos.</p>
}
```

Se activa **dentro del `else` que garantiza que `Model` no es nulo**. La condición consulta los datos existentes y decide si mostrar un mensaje: no altera el catálogo ni recalcula precios, y tampoco introduce persistencia.

Cuando el alumnado lo habilita en P1.5.2, comprobará que el contador sigue mostrando tres y se añade un texto nuevo. Esta separación entre datos y formato será la base de la vista parcial que aparecerá después.

1.5.3. El layout: la estructura compartida de la página

Si cada vista copiara su propio `<html>`, `<head>`, navegación, hojas CSS y pie, un cambio de diseño tendría que repetirse en todas las páginas. En MVC, el **layout** permite reunir esta estructura común.

Nuestro archivo `Views/Shared/_Layout.cshtml` contiene elementos como:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8" />
    <title>@ViewData["Title"] - Curso Web .NET 8</title>
</head>
<body>
    <nav><a href="/">Inicio</a><a href="/Productos">Productos</a></nav>
    <main>
        @RenderBody()
    </main>
    <footer>Curso ASP.NET Core MVC · .NET 8</footer>
</body>
</html>
```

Esta es una representación abreviada para explicar la idea; el archivo real incluye sus hojas de estilo, clases y otros enlaces. La instrucción `@RenderBody()` es especialmente importante: indica dónde se insertará el contenido de la vista concreta que devuelve la acción MVC.

El archivo `Views/_ViewStart.cshtml` establece:

```
@{
    Layout = "_Layout";
}
```

Por eso las vistas completas comparten el layout sin repetir en cada archivo la instrucción de asignación. El archivo `_ViewImports.cshtml`, en cambio, aporta directivas compartidas y el registro de Tag Helpers:

```
@using CursoWeb.Mvc
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Conviene evitar una confusión habitual: `_ViewStart.cshtml` interviene en las **vistas completas**, pero **no se ejecuta al representar una vista parcial**. Una parcial incluida dentro de otra página no introduce por sí sola un segundo layout. Esta distinción se volverá visible cuando añadamos tarjetas en 1.5.4.

Títulos que dependen de cada vista

Cada página puede indicar un valor mediante `ViewData["Title"]`:

```
@{
    ViewData["Title"] = "Detalle del producto";
}
```

La vista proporciona un valor y el layout lo utiliza dentro de `<title>`. Así el título del navegador cambia sin tener que crear varios layouts. Para datos más complejos, el modelo tipado es la opción principal; aquí `ViewData` se usa únicamente como mecanismo sencillo de comunicación de presentación con la estructura común.

1.5.4. Secciones de un layout: contenido opcional que depende de la página

Un layout también puede reservar una posición para contenido adicional que solo algunas vistas definen, mediante una **sección Razor**. Nuestro proyecto prepara este bloque comentado en el layout:

```
<aside id="notas-del-layout" aria-label="Notas de la página">
    @await RenderSectionAsync("NotasPagina", required: false)
</aside>
```

La instrucción `RenderSectionAsync("NotasPagina", required: false)` indica una posición para una sección de ese nombre y declara que **es opcional**. Como no es obligatoria, la portada y la lista no tienen que definirla.

En `Views/Productos/Detalle.cshtml`, otro bloque comentado permite definir:

```
@section NotasPagina
{
    <p id="nota-pagina-detalle">Detalle construido con una vista Razor tipada.</p>
}
```

Al activar ambos bloques, la página de detalle escribe su mensaje en el lugar dispuesto por el layout, y las demás vistas siguen funcionando. Esta es una forma de mantener la estructura compartida y permitir a cada página completar determinadas zonas.

Regla importante: declarar una sección no equivale a renderizarla en el mismo archivo. El layout debe renderizarla o ignorarla. En este ejercicio siempre activaremos **los dos bloques del paso 1.5.3**, de manera que la sección declarada en el detalle tiene un destino en el layout. El propósito es enseñar una combinación válida y reproducible; no se pide al alumno que active solo una de las dos mitades.

Una sección no debe confundirse con una vista parcial. La sección es contenido definido por la vista para ser colocado en una zona prevista del layout. Una parcial, en cambio, es un **archivo de marcado reutilizable** que una vista puede representar pasándole un modelo.

1.5.5. Vistas parciales: evitar repetir marcado

Supongamos que el mismo producto debe presentarse como tarjeta tanto en el catálogo como dentro de su propia ficha. Podríamos copiar el mismo `<article>` en ambos archivos, pero entonces cualquier cambio de presentación tendría que repetirse. Para evitarlo utilizaremos una **vista parcial**.

Una vista parcial es un archivo Razor `.cshtml` que genera un fragmento HTML para ser incluido dentro de otra vista. No requiere tener su propia acción ni dispone automáticamente de una página o layout independiente. Puede declarar su propio `@model` para trabajar con un tipo concreto.

Nuestro proyecto incorpora `Views/Shared/_TarjetaProducto.cshtml`, inicialmente así:

```
@model CursoWeb.Mvc.Models.Producto

<article class="tarjeta-producto" data-producto-id="@Model.Id">
    <h3>@Model.Nombre</h3>
    <p class="tarjeta-precio">Precio: @Model.Precio.ToString("0.00") €</p>
    <a href="/Productos/Detalle/@Model.Id">Ver detalle de @Model.Nombre</a>
</article>
```

El archivo real añade comentarios docentes y el bloque opcional del reto. La parcial recibe un solo `Producto`, presenta su nombre y precio y produce un enlace de detalle. Como se encuentra en `Views/Shared`, puede localizarse desde las vistas del catálogo y de la ficha por las convenciones de búsqueda de vistas.

Llamar a una parcial con el Partial Tag Helper

Para representar la tarjeta dentro de la página de listado utilizamos:

```
@foreach (var producto in Model)
{
    <partial name="_TarjetaProducto" model="producto" />
}
```

`<partial>` es un **Tag Helper**, habilitado por `_ViewImports.cshtml`. Su atributo `name` indica el nombre de la parcial; `model="producto"` le entrega el objeto actual de la iteración. El código Razor del archivo parcial accede a ese objeto como `Model`.

El Tag Helper construye el fragmento de HTML de forma asíncrona, sin que el alumnado tenga que gestionar ese detalle aquí. No es un componente Blazor ni crea interactividad por sí mismo. En M03 compararemos los componentes Razor de Blazor con las vistas de MVC, para no confundir conceptos que utilizan nombres y archivos parecidos.

Resultado: en el catálogo aparecen **tres tarjetas**, una por producto. La página ya mostraba un listado sencillo; activamos otra representación deliberadamente para poder comparar ambas y observar el uso de la parcial sin borrar el ejercicio anterior.

Reutilizar la misma parcial en el detalle

En `Views/Productos/Detalle.cshtml` se activa más adelante:

```
<section id="tarjeta-detalle" aria-label="Ficha presentada como tarjeta">
    <h2>Tarjeta reutilizada en el detalle</h2>
    <partial name="_TarjetaProducto" model="Model" />
</section>
```

Aquí `Model` es un único objeto `Producto` y se pasa directamente. La misma vista parcial produce una tarjeta, sin duplicar el código en el archivo de detalle.

La diferencia entre las dos invocaciones es intencionada:

- `model="producto"` recibe el objeto actual de un `foreach` del catálogo.
- `model="Model"` recibe el objeto completo de la vista de detalle.

El atributo `model` no debe confundirse con la directiva `@model`. El primero es un atributo del Tag Helper que pasa un objeto; el segundo declara el tipo esperado por el archivo Razor que va a representarlo.

Qué no debe hacer una parcial

Una vista parcial resulta apropiada para repetir marcado y lógica **simple de presentación**. No conviene ocultar dentro de ella acceso a una base de datos ni usarla como solución universal para construir páginas complejas. En esta práctica, la parcial **recibe datos**, y el controlador sigue siendo el que obtiene los productos.

Si se necesita lógica propia compleja para preparar un fragmento reutilizable, ASP.NET Core dispone también de otros mecanismos, como View Components. No forman parte del alcance de este capítulo.

1.5.6. El reto: una condición de precio centralizada en la parcial

El reto utiliza lo que acabamos de aprender: al modificar **una única vista parcial** debemos poder cambiar el HTML de la tarjeta en **dos páginas**.

Dentro de `Views/Shared/_TarjetaProducto.cshtml` existe un bloque de código comentado:

```
@if (Model.Precio < 3m)
{
    <strong class="etiqueta-economico">Precio económico</strong>
}
```

La condición compara el precio decimal del producto con tres unidades monetarias. El código del curso contiene estos valores: Cuaderno 4,50, Lápiz 1,20 y Carpeta 3,75. Por tanto, el mensaje aparece únicamente para **Lápiz**.

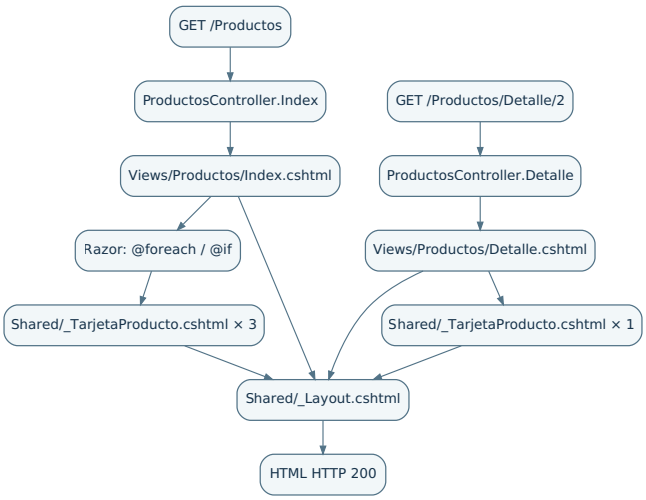
Esta expresión es apropiada en la parcial porque la usamos como una decisión **visual y local del ejercicio**; no estamos implementando una política real de descuentos ni alterando los precios. El umbral es ficticio y está explicado en el Markdown de prácticas.

Al activar el reto, la vista parcial añade la etiqueta a la tarjeta de Lápiz dentro del catálogo y dentro de `/Productos/Detalle/2`. La ficha de Cuaderno (`id=1`) no debe mostrarla. El archivo `Index.cshtml`, el archivo `Detalle.cshtml` y el controlador permanecen sin cambios adicionales.

Con ello se demuestra directamente el valor de la reutilización: **una modificación localizada produce un comportamiento coherente dondequiera que se represente la misma parcial**.

1.5.7. Flujo de composición de una respuesta HTML

Los elementos del ejercicio se relacionan de la siguiente manera:



El diagrama representa la **composición conceptual** del HTML. El motor Razor coordina la ejecución de vistas y parciales; no son servicios HTTP independientes ni se ejecuta un controlador distinto por cada tarjeta. La petición al navegador produce una respuesta HTML que contiene todos los fragmentos.

Cuando la página de detalle define la sección `NotasPagina`, el layout también coloca su contenido en la zona `RenderSectionAsync`. La parcial no necesita conocer esa sección ni declararla.

1.5.8. Errores habituales

«Una parcial es una página que el navegador solicita por separado». En este caso, no. Se representa dentro de otra vista del servidor. No necesitamos crear una acción ni una ruta HTTP para `_TarjetaProducto.cshtml`.

«Si llamo a `<partial>` desde un `foreach` se ejecuta el layout por cada elemento». No. El layout es el envoltorio de la vista completa; la parcial no ejecuta `_ViewStart.cshtml` ni inserta otro documento HTML completo.

«Puedo usar cualquier propiedad en `@Model` sin importar el tipo». No. La directiva `@model` declara qué tipo de datos recibirá el archivo. El listado y el detalle tienen modelos diferentes; la parcial espera un `Producto` individual.

«La sección del detalle aparece automáticamente con solo definir `@section`». No. El layout debe contener un marcador de representación con el nombre correspondiente, o la vista debe atender de otra forma el contrato de secciones. Activaremos ambos bloques juntos.

«Razor permite escribir C# en las vistas, por tanto allí va toda la lógica». No. Razor facilita decisiones de presentación; las operaciones de dominio o persistencia deben separarse de esas decisiones visuales.

«Si la parcial tiene código comentado, sus etiquetas pueden aparecer en HTML». Los comentarios Razor `@* ... *@` no se envían en el cuerpo HTML. Al retirarlos correctamente, el bloque comienza a generar contenido.

«El Tag Helper `<partial>` es un componente Blazor». No. Ambos pueden compartir sintaxis Razor, pero las vistas parciales MVC se renderizan como fragmentos de una respuesta HTML; los componentes Blazor tienen otro modelo de ejecución que veremos en M03.

1.5.9. Continuidad y alcance del módulo M01

La práctica 1.5 reproduce los conceptos en estados pequeños y observables: vistas tipadas ya existentes, condición Razor, layout con sección opcional, tarjeta parcial en el listado, reutilización de la tarjeta en el detalle y reto de etiqueta según precio. Cada estado se compila y se prueba contra Kestrel mediante HTTP real.

Con esta práctica se completa el **contenido funcional previsto de M01**: desde la primera petición hasta un catálogo con listados y fichas Razor reutilizables. Los **formularios, model binding de POST, validación con DataAnnotations y EF Core** se incorporarán en el M02, no en este módulo.

Para afianzar lo aprendido, recorre las cinco etapas del módulo con los proyectos incluidos y comprueba que cada cambio de código corresponde a un comportamiento observable. La práctica ofrece los procedimientos necesarios para continuar el desarrollo del catálogo.

Referencias técnicas

- Microsoft Learn, Razor syntax: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor?view=aspnetcore-8.0

- Microsoft Learn, layouts: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/layout?view=aspnetcore-8.0
- Microsoft Learn, partial views: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/partial?view=aspnetcore-8.0
- Microsoft Learn, Partial Tag Helper: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/tag-helpers/built-in/partial-tag-helper?view=aspnetcore-8.0